Recurrent Neural Network (RNN) – Google Stock Price Prediction Using LSTM

1. Aim of the Practical

The objective of this practical is to design a **time series prediction system** using a **Recurrent Neural Network (RNN)** to predict Google stock prices.

You used:

- Google stock dataset (GOOGL.csv)
- Deep Learning model: **LSTM (Long Short-Term Memory)**
- TensorFlow/Keras framework

The model learns patterns from past stock prices and predicts future prices.

---

2. What is Time Series Data?

A **time series** is data collected over time in sequence.

Examples:

- Stock prices
- Weather data
- Temperature readings
- Sales forecasting

In your dataset:

Date	Open Price
2004-08-19	50.05
2004-08-20	50.55
2004-08-23	55.43

Each row depends on previous rows.

---

3. What is RNN?

A **Recurrent Neural Network (RNN)** is a neural network specially designed for sequential data.

Unlike normal neural networks:

- RNN remembers previous information
- It uses past outputs to influence future outputs

Example

To predict today's stock price:

- Yesterday's price matters
- Last week's trend matters

RNN can remember this sequence.

---

4. Problem with Basic RNN

Normal RNN suffers from:

- **Vanishing Gradient Problem**
- Cannot remember long-term dependencies

Example:

- It may forget stock trends from many days ago.

To solve this:

LSTM is used.

---

5. What is LSTM?

LSTM = **Long Short-Term Memory**

It is an advanced type of RNN that:

- Remembers important information for a long time
- Forgets unnecessary information

Very useful for:

- Stock prediction
- NLP
- Speech recognition
- Time series forecasting

---

6. Dataset Used

You used Google stock dataset:

Column	Meaning
Open	Opening stock price
High	Highest price
Low	Lowest price

Column	Meaning
Close	Closing price
Volume	Number of shares traded

---

7. Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Purpose

- pandas → data handling
- numpy → numerical operations
- matplotlib → graphs

---

8. TensorFlow/Keras Libraries

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
```

Purpose

Layer	Use
Sequential	Creates neural network
LSTM	RNN memory layer
Dropout	Prevents overfitting
Dense	Output layer

---

9. Loading Dataset

```
df = pd.read_csv('GOOGL.csv')
```

Loads Google stock data into dataframe.

---

10. Selecting Training Data

training_Set = df.iloc[:,1:2].values

What does this mean?

- iloc[:,1:2]
- Selects only **Open price** column

Why?
Because model predicts stock price using historical opening prices.

---

11. Feature Scaling using MinMaxScaler

scaler = MinMaxScaler(feature_range=(0,1))

training_set_scaled = scaler.fit_transform(training_Set)

Why Scaling is Needed?

Stock prices are large numbers:

- 50
- 1000
- 2800

Neural networks train better on small values.

So scaling converts data between:

[
0 \text{ to } 1
]

Formula

X_{scaled}=\frac{X-X_{min}}{X_{max}-X_{min}}

---

12. Creating Time Steps (60 Days Window)

```
for i in range(60,len(training_set_scaled)):
    x_train.append(training_set_scaled[i-60:i,0])
    y_train.append(training_set_scaled[i,0])
```

MOST IMPORTANT CONCEPT

The model uses:

- Previous 60 days prices
- To predict next day price

Example

Input:

Day	Price
1	100
2	101
...	...
60	120

Output:

- Predict Day 61 price

This is called:

Sliding Window Technique

---

13. Converting List to NumPy Array

```
x_train = np.array(x_train)

y_train = np.array(y_train)
```

TensorFlow requires NumPy arrays.

---

14. Reshaping Data for LSTM

```
x_train = np.reshape(

    x_train,

    (x_train.shape[0],x_train.shape[1],1)

)
```

VERY IMPORTANT THEORY

LSTM requires 3D input.

Format:

```
[
(samples,\ time\ steps,\ features)
]
```

Your Shape

Dimension	Meaning
samples	Number of training examples
60	Previous 60 days
1	One feature (Open price)

Example Shape

(4371, 60, 1)

Meaning:

- 4371 training samples
- Each sample contains 60 days
- Each day has 1 feature

---

15. Building LSTM Model

model = Sequential()

Creates empty neural network.

---

16. First LSTM Layer

```
model.add(
   LSTM(
      units=50,
      return_sequences=True,
      input_shape = (x_train.shape[1],1)
   )
)
```

Explanation

units=50

Number of neurons in LSTM layer.

More units:

- Better learning
- More computation

---

return_sequences=True

Important when stacking multiple LSTM layers.

Why?

First LSTM layer sends sequence output to next LSTM layer.

---

input_shape

(60,1)

Means:

- 60 previous days
- 1 feature per day

---

17. Dropout Layer

```
model.add(Dropout(0.2))
```

Purpose

Prevents overfitting.

What happens?

20% neurons are randomly ignored during training.

This improves generalization.

---

18. Second LSTM Layer

```
model.add(
    LSTM(
        units=50,
        return_sequences=False
    )
)
```

return_sequences=False

Now only final output is needed.

Because:

- Next layer is Dense layer

- Not another LSTM layer

---

19. Output Layer

```
model.add(Dense(units=1))
```

Only one value predicted:

- Next stock price

Since this is regression:

- No softmax/sigmoid used

---

20. Compiling the Model

```
model.compile(
    optimizer='adam',
    loss='mean_squared_error'
)
```

Optimizer: Adam

Adam updates weights efficiently.

Very commonly used optimizer.

Loss Function: MSE

Used for regression problems.

Formula:

$$MSE=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

Where:

- y_i = actual price
- $\hat{y}_i$ = predicted price

Smaller MSE = better model.

---

21. Training the Model

```
model.fit(
    x_train,
    y_train,
```

```
    epochs=20,
    batch_size=32
)
```

Epochs

One complete pass through dataset.

You used:

epochs=20

Meaning:

- Model learned 20 times from entire data.

Batch Size

batch_size=32

Model processes:

- 32 samples at one time

---

22. Training Output Interpretation

loss: 0.0106

loss: 0.0009

Loss decreases over epochs.

This means:

- Model is learning successfully.

---

23. Predicting Stock Prices

predicted_stock_pprice = model.predict(x_train)

Model predicts stock prices.

Output is still scaled between:
[
0 \text{ and } 1
]

---

24. Inverse Transform

predicted_stock_price = scaler.inverse_transform(predicted_stock_pprice)

Converts scaled values back to original stock prices.

Example:

- 0.85 → 2700 dollars

---

25. Plotting Graph

plt.plot(real_prices)

plt.plot(predicted_prices)

Red Line

Actual stock prices.

Blue Line

Predicted stock prices.

If both lines are close:

- Model performance is good.

---

26. Understanding the Graph

Good Prediction

- Blue line follows red line closely.

Bad Prediction

- Large gap between lines.

Your model performs reasonably well because:

- Loss became very small.

---

27. Why LSTM is Better for Stock Prediction?

Because stock prices depend on:

- Previous trends
- Long-term patterns
- Historical sequences

LSTM remembers these dependencies.

---

28. Applications of RNN/LSTM

Application	Use
Stock prediction	Financial forecasting
Weather forecasting	Temperature prediction
NLP	Language translation
Speech recognition	Voice assistants
Chatbots	Sequential text processing

---

29. Important Viva Questions

Q1. Why use LSTM instead of ANN?

ANN cannot remember sequence information.
LSTM remembers past data.

Q2. Why scaling is important?

Large values slow training and reduce accuracy.

Q3. Why reshape data into 3D?

LSTM expects input format:
[
(samples,\ time\ steps,\ features)
]

Q4. Why use Dropout?

To reduce overfitting.

Q5. Why use MSE loss?

Because this is a regression problem.

Q6. What does return_sequences do?

- True → passes sequence to next LSTM
- False → passes final output only

---

30. Final Conclusion

In this practical:

- Google stock data was used
- Data was normalized
- Time sequences of 60 days were created
- LSTM-based RNN model was built
- Model successfully predicted stock prices
- Predictions were visualized using graphs

The project demonstrates how deep learning can perform time series forecasting using sequential historical data.